Assignment 2 Report

Image Filtering

In image_filtering, I found that edge detection kernels highlight the edges more. That is, the lines having the edges are brighter, while other shapes get darker. An image usually has a lot of information, and in order for our algorithm to start classifying it, it must break the problem into smaller dimensions, smaller features, and smaller details. Edge detection is one of the features it extracts to reduce the complexity of an image.

Another tool to reduce the complexity of an image is using blur on an image. That is, taking an n x m pixels and producing one pixel based on the average of the n x m pixels. Many images have a high resolution, which can take up the training and testing time of our models. When I used a 5 x 5 kernel for blurring, the output of the image was more blurry than the 3x3 and 2x2. Although we may lose detail in the image, the buildings stood out more since that is the largest object in the image. This also reduces the reflection on the buildings and texture on the objects. This can be useful when we want to extract the large objects in an image.

For my custom image, I used to put it through a series of filters. I used Disneyland's Pixar Hotel Lobby, which has a lot of different shapes and lightning effects. I then implemented the Sobel Horizontal and Vertical filters. I noticed that not only does it highlight the edges, but it also highlights the depth of the objects in the image. As opposed to the regular horizontal edge filter from the buildings, the Sobel Horizontal filter produces an image that still retains some depth. You can see this difference in the building image filters where I also added the Sobel filter there too.

Another filter I used was the corner detection filter. This filter highlighted corners aggressively. This filter also highlighted quadrilaterals especially. Although I used four different filters, like top left, top right, bottom left, and bottom right corners, I still somehow managed to produce similar output pictures for all four.

When combining filters, you can see that a clearer object shape is more outlined as you can see in the last image. I used a blurring filter together with a horizontal

Sobel filter. Together, the filter has less reflections and less artifacts produced by the light effects.

Convolutional Neural Network Build Model

The table below shows the parameters I used to train the model and its results.

First Layer Input	Optimizer	Number of Layers	Learning Rate	Training Time (mm:ss)	Total Accuracy
16	SGD	2	0.01	2:00	57%
16	Adam	2	0.01	2:03	45%
16	SGD	2	0.001	2:00	55%
16	Adam	2	0.001	2:04	54%
16	SGD	2	0.0001	1:54	55%
16	Adam	2	0.0001	1:57	56%
16	SGD	3	0.01	2:34	54%
16	Adam	3	0.01	2:38	37%
16	SGD	3	0.001	2:31	43%
16	Adam	3	0.001	2:40	49%
32	SGD	3	0.01	6:04	57%
32	Adam	3	0.01	6:26	31%
32	SGD	3	0.001	5:57	28%
32	Adam	3	0.001	6:07	73%
32	SGD	3	0.0001	6:50	74%
32	Adam	3	0.0001	6:59	77%
64	Adam	3	0.001	13:33	73%
64	Adam	3	0.0001	13:19	77%
16	Adam	4	0.001	3:18	70%
32	Adam	4	0.001	8:28	73%
16	Adam	4	0.0001	3:24	54%
32	Adam	4	0.0001	9:25	62%

A pattern I observed was that when using the SGD optimizer, I sometimes get good prediction results that come close to the Adam optimizer, but when that was not the case, its prediction result could be way off than the Adam optimizer. One of the reasons is that the SGD optimizer could overshoot from a local minima during gradient descent and be off the global minima with no way of coming back. Adam could have a similar behavior, but it is highly less likely.

Another pattern is the correlation between learning rate and the number of convoluted layers. The more layers a model had, the better it is to have a smaller learning rate. This has the opposite effect when there are fewer layers and a smaller learning rate; this gave a lower overall percentage of correct predictions.

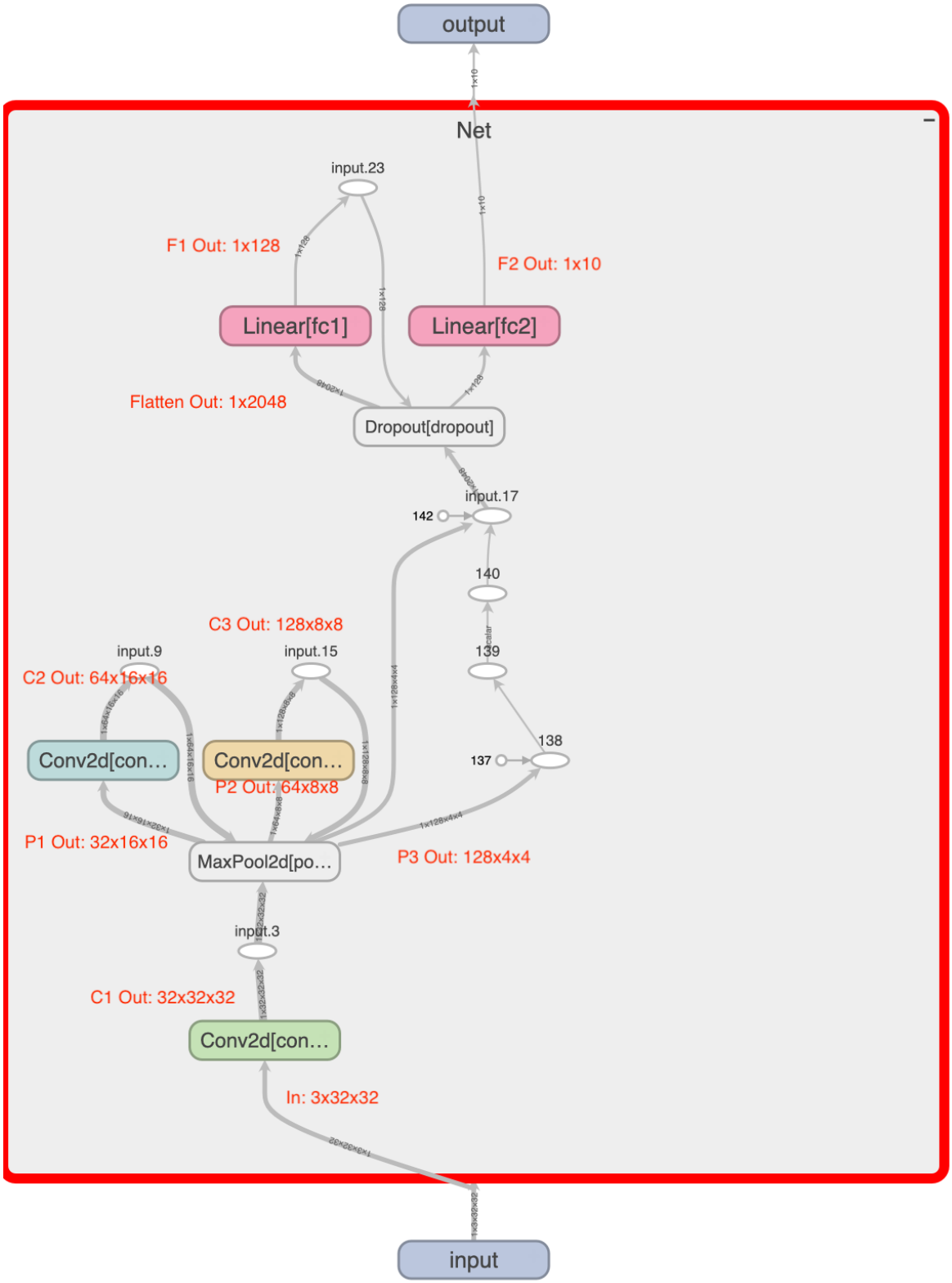
I also noticed that when I had a lower input parameter like 16 to the first convoluted layer, the better it performed with less total amount of layers. It gradually decreased its accuracy as I added more layers.

An easy observation is that the training time increased as both the input parameters increased for the convoluted layers with the amount of total layers. That is because the amount of parameters increased based on these two values.

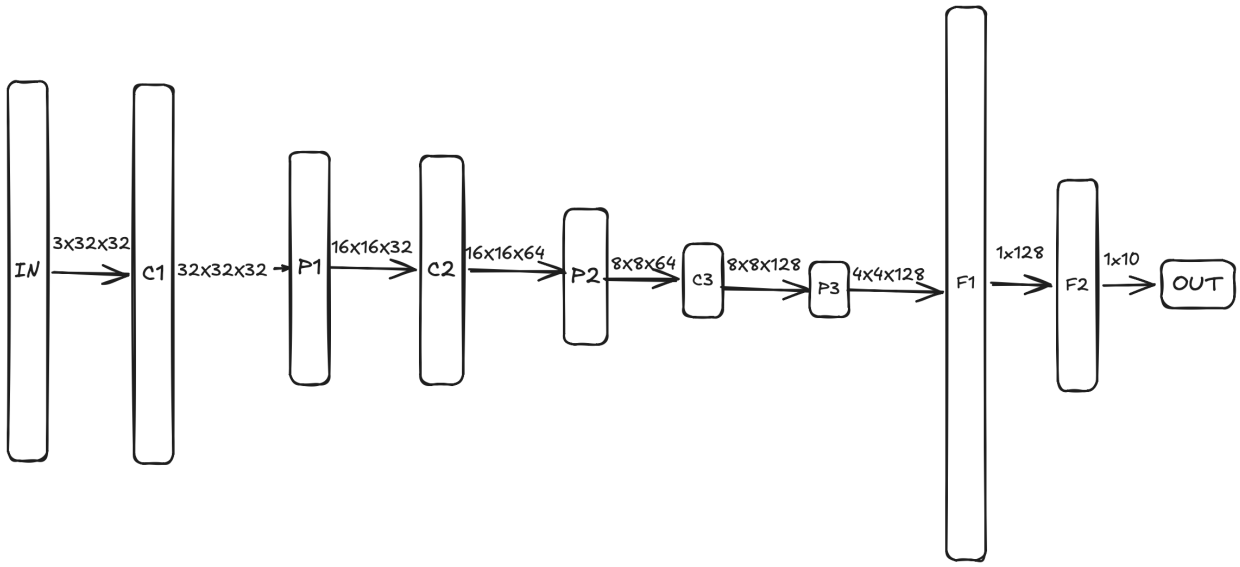
For my final and optimal solution, I used the architecture with 32 starting inputs and having 3 convoluted layers. The reason I chose this architecture as opposed to the one with 64 starting inputs, even though having the same prediction score was that it was more efficient. That is, it had less amount of total parameters, and its training time was half of the 64 input.

An important note is that I got different prediction values rerunning the program due to random images being trained on and initial weights being different, so your output may be different but similar in the range of 5% difference. The number of epochs may need to be added if the model gets a lower prediction rate.

Below are the diagrams for the CNN Architecture built by PyTorch and one by me using Excalidraw.



Tensorboard CNN Architecture Diagram



My CNN(Excalidraw) Architecture Diagram